EXPERIMENT 10

Transformation-Based (Brill) Tagging using Simple Transformation Rules

AIM

To implement transformation-based error-driven Part-of-Speech tagging (Brill's tagging method) using inductive transformation rules in Python.

PROCEDURE

1. Initialize all tokens with a baseline tagger (e.g. most frequent tag: NOUN).
2. Define contextual transformation rules (e.g., 'Change NOUN to VERB if preceded by modal/determiner', 'Change VERB to NOUN if preceded by DET').
3. Sequentially apply the transformation rules to correct classification errors.
4. Track tags before and after transformation to observe error reductions.
5. Display the comparative output demonstrating rule transitions.

PROGRAM

```
class TransformationTagger:
    def __init__(self):
        self.lexicon = {
            "the": "DT", "a": "DT", "an": "DT",
            "can": "NN", "fish": "NN", "water": "NN",
            "will": "NN", "book": "NN", "flight": "NN"
        }
        self.rules = [
            ("WORD_IN_MODALS_AND_NEXT_IS_NN", "NN", "MD"),
            ("PREV_IS_MD", "NN", "VB"),
            ("PREV_IS_DT", "VB", "NN")
        ]

    def initial_tag(self, tokens):
        return [self.lexicon.get(tok.lower(), 'NN') for tok in tokens]

    def apply_transformations(self, tokens):
        tags = self.initial_tag(tokens)
        history = [list(tags)]

        for i in range(len(tokens) - 1):
            if tokens[i].lower() in ['can', 'will'] and tags[i] == 'NN':
                tags[i] = 'MD'

        history.append(list(tags))

        for i in range(1, len(tokens)):
            if tags[i-1] == 'MD' and tags[i] == 'NN':
                tags[i] = 'VB'
```

```
        history.append(list(tags))
        return tags, history

tagger = TransformationTagger()
sentence = "The fish can fish a fish".split()
final_tags, history = tagger.apply_transformations(sentence)

print("Sentence:", " ".join(sentence))
print("-" * 50)
print(f"Step 0 (Initial Baseline): {list(zip(sentence, history[0]))}")
print(f"Step 1 (After Modal Rule): {list(zip(sentence, history[1]))}")
print(f"Step 2 (After Context Rule): {list(zip(sentence, history[2]))}")
print("-" * 50)
print("Final Transformation Result:")
for w, t in zip(sentence, final_tags):
    print(f"  {w:<8} -> {t}")
```

OUTPUT

```
> python3 experiment_10.py
--------------------------------------------------------------------------------
Sentence: The fish can fish a fish
--------------------------------------------------
Step 0 (Initial Baseline): [('The', 'DT'), ('fish', 'NN'), ('can', 'NN'), ('fish',
'NN'), ('a', 'DT'), ('fish', 'NN')]
Step 1 (After Modal Rule): [('The', 'DT'), ('fish', 'NN'), ('can', 'MD'), ('fish',
'NN'), ('a', 'DT'), ('fish', 'NN')]
Step 2 (After Context Rule): [('The', 'DT'), ('fish', 'NN'), ('can', 'MD'), ('fish',
'VB'), ('a', 'DT'), ('fish', 'NN')]
--------------------------------------------------
Final Transformation Result:
  The      -> DT
  fish     -> NN
  can      -> MD
  fish     -> VB
  a        -> DT
  fish     -> NN
[Process completed with exit code 0]
```

RESULT

Thus, the transformation-based tagging system applying contextual disambiguation rules was successfully implemented and verified.